nb_p2_batch_job_1

Bronze to Silver: Batch Job 1

Technical documentation of the notebook: what it does, how each part is implemented, and the tables it produces

Project: Fleet Telematics Lakehouse on Microsoft Fabric | SIC 8 / 2026 | Big Data Engineering, Project 3

Phase 2, Silver layer | Last updated: 2026-10-28

1. Purpose and scope

nb_p2_batch_job_1 is the batch job that turns the raw telemetry stored in Bronze into clean, typed, enriched Silver tables. It loops over four signals: **gps**, **engine**, **fuel** and **temperature**. For every signal it reads only the new Bronze rows (watermark window), validates each message with the `fleet_shared` library, sends rejected messages to a dead-letter queue (DLQ), enriches the valid ones, removes duplicates, adds quality flags, merges the result into `silver.silver_<signal>`, records the counts in an ingestion log and moves the watermark. When all signals are done, **Gate 1** computes the share of invalid rows and returns a JSON status as the notebook exit value.

Inputs and outputs

Direction	Object	Content
Input	bronze.bronze_<signal>_raw	Raw message text (`raw_value`) plus `ingestion_time`, `source_type`, `source_file`, `file_line`
Input	ref.ref_vehicles_raw	Vehicle versions with `active_from`, `active_to`, `updated_at`, `fuel_tank_capacity_liters`
Input	ref.ref_trips	Trip reference: `trip_id`, `route_id`, `driver_id`, `cargo_type_id`
Input	Files/evidence/run_manifest.json, run_summary.json	Run timing (`sim_start`, `wall_start`, acceleration factor) and the message counts of the run
Input	fleet_shared 1.0.0 (environment env_fleet)	Message validation, field definitions, error codes and flag names
Output	silver.silver_gps / engine / fuel / temperature	Validated, typed and enriched rows, Delta tables partitioned by `event_date`
Output	ops.dlq_events	Rejected messages with the error code (stage BATCH)
Output	ops.ops_ingestion_log	One row per signal per run, plus one Gate 1 row per run
Output	ops.ops_watermark	Last processed Bronze ingestion time per signal
Output	Notebook exit value	Gate 1 JSON status

2. Processing flow

The function `process_signal(signal)` runs the steps below in this order. `main()` calls it for each signal and then runs Gate 1.

#	Step	Function	Result
1	Select the window of new rows	get_window	lower = saved watermark, upper = newest Bronze ingestion_time
2	Validate every row	load_lookups, make_validate_udf, validate	good rows and bad rows
3	Check the vehicle is active on the event date	split_by_vehicle_activity	inactive-vehicle rows move to the bad rows
4	Write bad rows to the DLQ	write_dlq	rows in ops.dlq_events

#	Step	Function	Result
5	Parse, enrich and compute lateness	parse_and_enrich	typed columns, dq_flags text, route/driver/cargo, lateness_sec
6	Remove duplicates	dedup_batch	one row per event_id, duplicates counted
7	Add sequence quality flags	add_sequence_flags	SPIKE, FLATLINE, IMPOSSIBLE_JUMP appended to dq_flags
8	MERGE into Silver	merge_silver	only event_ids not yet in Silver are inserted
9	Log the counts	write_log	row in ops.ops_ingestion_log
10	Move the watermark	set_watermark	watermark = upper (always the last step)
11	Gate 1 and exit value	run_gate1, write_gate_log, main	JSON status in the log and as the exit value

3. Notebook walkthrough

3.1 Imports and configuration

The session time zone is set to UTC so that timestamps match Kusto and Bronze. The job name, the timestamp format used in SQL literals and the watermark text format are defined here. The watermark text keeps microseconds, so no Bronze row is reloaded.

```
spark.conf.set("spark.sql.session.timeZone", "UTC")
JOB_NAME = "nb_p2_batch_job_1"
TS_FMT = "%Y-%m-%d %H:%M:%S.%f"      # SQL TIMESTAMP literals
WM_FMT = "%Y-%m-%dT%H:%M:%S.%fZ"     # watermark text (microseconds kept)
```

3.2 Run manifest and job settings

`load_manifest()` reads `Files/evidence/run_manifest.json` and keeps `sim_start` and `wall_start` (as epoch seconds), the acceleration factor **A** and `fast_mode`. `check_manifest_against_bronze()` compares `per_signal` in `run_summary.json` with the Bronze row counts and prints OK or MISMATCH per signal, and prints a warning if the manifest does not belong to the data or if the run used fast mode. The notebook then prints the library defaults (`fs.DEFAULT_RULES`).

Job settings

Setting	Value	Used for
ENABLE_ACTIVE_CHECK	True	Reject rows whose vehicle is not active on the event date
SPIKE_DELTA_C	8.0	Temperature change from the previous reading above this value gives SPIKE
FLATLINE_N	10	This many identical consecutive temperature readings give FLATLINE
MAX_JUMP_SPEED_KMH	200.0	Implied GPS speed between two consecutive points above this gives IMPOSSIBLE_JUMP
GATE_MAX_INVALID_PCT	5.0	Gate 1 fails above this share of invalid rows (quality_gate_max_invalid_pct)

3.3 Window selection: get_window

For each signal the lower bound is the watermark stored in `ops.ops_watermark` and the upper bound is the newest `ingestion_time` in the Bronze table. Only rows with `lower < ingestion_time <= upper` are processed. If the window is empty, the signal prints "nothing to do" and returns.

```
lower = spark.sql("SELECT to_timestamp(watermark_utc) AS w FROM ops.ops_watermark WHERE signal='gps'").first()["
upper = spark.table("bronze.bronze_gps_raw").agg(F.max("ingestion_time")).first()[0]
```

3.4 Lookups and validation

`load_lookups()` builds three small reference structures that are shipped to the executors inside the validation UDF (a broadcast by closure):

- **known_vehicles**: the set of vehicle ids in `ref_vehicles_raw`.
- **known_trips**: the set of trip ids in `ref_trips`.
- **tank_cap**: one fuel tank capacity per vehicle, taken from the latest row by `updated_at`; a fuel level above the capacity is rejected as OUT_OF_RANGE.

`make_validate_udf()` wraps `fleet_shared.validate_message()` in a Spark UDF that returns the struct `(ok, error_code, error_message, retryable, flags, record_json)`. The library also receives now, the simulated time used for the future-timestamp check. `simulated_now(upper)` computes it from the manifest: `sim_start + (upper - wall_start) * A`, returned as a naive datetime because the library compares it with naive datetimes.

`validate()` reads the window from Bronze, applies the UDF, caches the result, counts the Bronze rows and splits them into good rows (`v.ok = true`) and bad rows (`v.ok = false`). Unknown vehicles and trips are detected by the lookups (UNKNOWN_VEHICLE, UNKNOWN_TRIP).

3.5 Vehicle active on the event date

After the library validation, good rows go through `split_by_vehicle_activity()` (switch `ENABLE_ACTIVE_CHECK`). A row is kept only if at least one version of its vehicle in `ref_vehicles_raw` is active on the event date: `active_from <= event date <= active_to`, where an empty or NULL `active_to` means the vehicle is still active. The check is a broadcast semi join (active rows) and anti join (inactive rows) on the vehicle id and the date range. Inactive rows are added to the bad rows with error code **UNKNOWN_VEHICLE** and the message "Vehicle not active at event time".

```
cond = ((col("_veh") == col("iv_vehicle")) &
        (col("iv_from").isNull() | (col("iv_from") <= col("_ev_date"))) &
        (col("iv_to").isNull()   | (col("_ev_date") <= col("iv_to"))))
active   = g.join(iv, cond, "left_semi")
inactive = g.join(iv, cond, "left_anti")
```

3.6 Dead-letter queue: write_dlq

Rejected rows are written to `ops.dlq_events` with stage **BATCH**, the error code and message, the `event_id` (read with `get_json_object`, which returns NULL instead of failing on broken JSON), the raw text and the Bronze arrival time. The write is **idempotent**: before inserting, the function deletes the DLQ rows of the same signal, stage and Bronze time window, so a re-run never doubles them.

```
DELETE FROM ops.dlq_events
WHERE signal = '<signal>' AND stage = 'BATCH'
  AND bronze_ingestion_time > TIMESTAMP'<lower>' AND bronze_ingestion_time <= TIMESTAMP'<upper>'
```

3.7 Parse, enrich and lateness: parse_and_enrich

Good rows carry the cleaned record as JSON text. The function:

- builds a DDL schema from the library field list (`fs.SIGNAL_FIELDS[signal]`) plus the common fields, so the same code works for every signal, and parses the JSON with `from_json`;
- stores `dq_flags` as comma-separated text (for example `UNIT_CONVERTED,SPIKE`), or NULL when no flag is present;
- converts `event_time` to a timestamp and derives `event_date` (the partition column);
- joins `ref_trips` (broadcast) on `trip_id` to add `route_id`, `driver_id` and `cargo_type_id`;
- computes `lateness_sec` (below).

lateness_sec. `event_time` is simulated time and `ingestion_time` is wall-clock time, so they cannot be subtracted directly. The simulated time is first mapped to the wall-clock time at which the message was expected to arrive, using the values of the run manifest, and the expected arrival is then subtracted from the real ingestion time:

```
expected_arrival = wall_start + (event_time - sim_start) / A
lateness_sec     = ingestion_time - expected_arrival        # seconds, rounded to 3 decimals
```

3.8 Deduplication: dedup_batch

Within the batch the function keeps one row per `event_id` (the library `DEDUP_KEY`): a window partitioned by `event_id` and ordered by `ingestion_time` ascending keeps the first copy that arrived. It returns the deduplicated DataFrame, the number of rows kept and the number of duplicates removed; the removed count goes to the ingestion log.

3.9 Sequence quality flags: add_sequence_flags

Some flags need the previous reading of the same vehicle. They are computed on the deduplicated batch with window functions and appended to `dq_flags` with `concat_ws(',', ...)`, so the column stays a single text value.

Flag	Signal	Rule
`SPIKE`	temperature	Absolute difference to the previous reading of the same vehicle and sensor (ordered by event_time, sequence_number) is above SPIKE_DELTA_C
`FLATLINE`	temperature	The last FLATLINE_N readings of the same vehicle and sensor are all present and identical (min = max)
`IMPOSSIBLE_JUMP`	gps	Haversine distance to the previous point divided by the time gap is above MAX_JUMP_SPEED_KMH (time gap must be positive)

Engine and fuel have no sequence flags. The previous row is looked up inside the current batch.

3.10 MERGE into Silver: merge_silver

The deduplicated rows are selected in the column order of the target table, registered as the temporary view `batch_silver` and merged. Only `event_ids` that are not in Silver yet are inserted, so repeating a run inserts nothing.

```
MERGE INTO silver.silver_<signal> t
USING batch_silver s ON t.event_id = s.event_id
WHEN NOT MATCHED THEN INSERT *
```

3.11 Logging and watermark: write_log, set_watermark

`write_log` appends one row to `ops.ops_ingestion_log` per signal per run with the window, the Bronze, Silver, DLQ and duplicate counts and the status (SUCCESS or FAILED with the error text). `set_watermark` updates `ops.ops_watermark` to the upper bound and is always the last step of a successful run.

3.12 Orchestration of one signal: process_signal

`process_signal` chains the steps of section 2. If any step raises an exception, a FAILED row is logged, the exception is re-raised and the **watermark is not moved**, so the same window is processed again on the next run. The cached DataFrame is always un-persisted in the `finally` block. At the end it prints the balance check **bronze = silver + dlq + duplicates removed** and returns the four counts.

```
validated, good, bad, n_bronze = validate(signal, lower, upper, lookups)
n_dlq = write_dlq(bad, signal, batch_id, lower, upper)
enriched = parse_and_enrich(good, signal)
deduped, n_silver, n_dups = dedup_batch(enriched)
deduped = add_sequence_flags(deduped, signal)
merge_silver(deduped, signal)
write_log(...); set_watermark(signal, upper)        # watermark last
```

3.13 Gate 1 and main()

`main()` runs `process_signal` for gps, engine, fuel and temperature, prints a balance table, runs Gate 1 and writes its status to the log.

Gate 1 (run_gate1). Invalid share = DLQ rows / Bronze rows, computed overall and per signal over the signals processed in the run. The status is **FAIL** when the overall share or the share of any signal is above `GATE_MAX_INVALID_PCT` (5 percent), otherwise **PASS**. Silver is written in both cases; the gate only tells the caller whether to continue. If no signal had new data, the status is PASS with the note "no new data".

Gate log row (write_gate_log). The same JSON is stored in `ops.ops_ingestion_log` with stage **GATE_1** and signal **ALL** (JSON in `error_message`, PASS or FAIL in `status`). The count columns are left NULL on purpose, so that sums over stage BATCH are not counted twice.

Gate 1 JSON fields

Field	Meaning
`status`	PASS or FAIL
`invalid_pct`	Overall invalid share in percent (DLQ rows / Bronze rows * 100)
`threshold_pct`	The limit used (5.0)
`per_signal`	Invalid share in percent for each processed signal
`failed_signals`	Signals above the limit (empty list when none)
`rows_bronze, rows_dlq`	Totals over the processed signals

3.14 Exit value

The last cell returns the Gate 1 JSON as the notebook exit value with `mssparkutils.notebook.exit(json.dumps(gate))`, so a pipeline or Airflow can read it and decide whether the jobs after this notebook run. It ends the notebook run, so it is the last cell.

3.15 Verification cells

The notebook contains queries used to check each run:

- event_id uniqueness, NULL keys and NULL event times in a Silver table;
- DLQ counts per signal and error code;
- enrichment completeness (NULL route, driver or cargo, trip ids missing from ref_trips, trip_id uniqueness in ref_trips);
- `lateness_sec` statistics per Silver table (rows, non-NULL rows, min, median, 99th percentile, max);
- `dq_flags` value counts for gps and temperature;
- an isolated test that sends a message with the unknown vehicle V099 through `validate_message` and expects UNKNOWN_VEHICLE (writes nothing).

4. Silver tables

Four Delta tables, `silver.silver_gps`, `silver_engine`, `silver_fuel` and `silver_temperature`, all partitioned by `event_date`. Every table has the same 14 common columns followed by the columns of its signal. The key is `event_id`.

4.1 Common columns (all four tables)

Column	Type	Purpose
`event_id`	STRING	Unique id of the message. Primary key of the Silver tables, MERGE key and duplicate key
`vehicle_id`	STRING	Vehicle that sent the message (checked against ref_vehicles_raw)
`trip_id`	STRING	Trip the vehicle was on. Join key to ref_trips
`device_id`	STRING	Telematics device that sent the message (for example DEV_V001)
`event_time`	TIMESTAMP	When the reading was taken (simulated time, UTC)
`event_date`	DATE	Date part of event_time. Partition column
`sequence_number`	BIGINT	Per-device message counter; orders readings and reveals gaps
`schema_version`	INT	Version of the message schema (1)
`ingestion_time`	TIMESTAMP	When the row was ingested into Bronze (wall-clock, UTC); basis of the watermark and of lateness_sec
`lateness_sec`	DOUBLE	Seconds between the expected arrival (from the run manifest) and the real ingestion time
`dq_flags`	STRING	Comma-separated data-quality flags, for example UNIT_CONVERTED,SPIKE. NULL when no flag is present
`route_id`	STRING	Route of the trip (from ref_trips)
`driver_id`	STRING	Driver of the trip (from ref_trips)
`cargo_type_id`	STRING	Cargo type of the trip, for example DRY_GOODS, MEDICINE_COLD, FROZEN_FOOD, FOOD_CHILLED (from ref_trips)

4.2 silver_gps (19 columns: 14 common + 5)

Column	Type	Purpose
`latitude`	DOUBLE	Latitude of the vehicle in decimal degrees
`longitude`	DOUBLE	Longitude of the vehicle in decimal degrees
`speed_kmh`	DOUBLE	Ground speed reported by the GPS, km/h
`heading_degrees`	DOUBLE	Direction of travel in degrees
`gps_accuracy_meters`	DOUBLE	Reported position accuracy in meters (larger is worse)

4.3 silver_engine (19 columns: 14 common + 5)

Column	Type	Purpose
`engine_on`	BOOLEAN	Whether the engine is running
`engine_rpm`	INT	Engine speed in revolutions per minute
`engine_temperature_c`	DOUBLE	Engine temperature in degrees Celsius
`vehicle_speed_kmh`	DOUBLE	Vehicle speed from the engine unit, km/h
`battery_voltage_v`	DOUBLE	Battery voltage in volts

4.4 silver_fuel (18 columns: 14 common + 4)

Column	Type	Purpose
fuel_level_liters	DOUBLE	Fuel in the tank, liters (checked against the vehicle tank capacity)
trip_fuel_used_liters	DOUBLE	Fuel used during the trip, liters
odometer_km	DOUBLE	Odometer reading, km
is_refueling	BOOLEAN	Whether the vehicle is refueling at this reading

4.5 silver_temperature (20 columns: 14 common + 6)

Column	Type	Purpose
shipment_id	STRING	Shipment carried by the vehicle
sensor_id	STRING	Temperature sensor that produced the reading
temperature_value	DOUBLE	Measured temperature in degrees Celsius (Fahrenheit readings are converted and flagged UNIT_CONVERTED)
temperature_unit	STRING	Unit of temperature_value after validation (C)
setpoint_temperature_c	DOUBLE	Target temperature of the cold chain, degrees Celsius
sensor_status	STRING	Sensor health reported by the device (for example OK)

5. Supporting tables

5.1 ops.dlq_events

Column	Type	Purpose
dlq_id	STRING	Unique id of the DLQ row (uuid)
batch_id	STRING	Id of the run that rejected the row (links to ops_ingestion_log)
stage	STRING	Pipeline stage that rejected the row (BATCH for this notebook)
signal	STRING	gps, engine, fuel or temperature
error_code	STRING	Reason for the rejection (see section 6.2)
error_message	STRING	Detail text from the validation
event_id	STRING	Event id read from the raw text when it can be read, otherwise NULL
raw_value	STRING	The exact raw message text, kept for replay and analysis
source_type, source_file, file_line	as in Bronze	Origin of the Bronze row (stream or file, file name and line)
bronze_ingestion_time	TIMESTAMP	Ingestion time of the Bronze row; used for the window and for idempotent deletes
is_retryable	BOOLEAN	Whether the error can disappear later (for example a vehicle added to the reference data)
created_at	TIMESTAMP	When the DLQ row was written

5.2 ops.ops_ingestion_log

Column	Type	Purpose
log_id	STRING	Unique id of the log row
batch_id	STRING	Run id of one signal's processing (NULL on the Gate 1 row)
job_name	STRING	nb_p2_batch_job_1
signal	STRING	Signal processed, or ALL on the Gate 1 row
stage	STRING	BATCH for signal runs, GATE_1 for the gate row
source_file	STRING	Source file (file ingestion only; NULL here)
window_start_utc, window_end_utc	STRING	Lower and upper bound of the processed Bronze window (ISO text)
rows_bronze	BIGINT	Bronze rows in the window
rows_silver	BIGINT	Rows kept after validation and deduplication
rows_dlq	BIGINT	Rows sent to the DLQ
rows_duplicates_removed	BIGINT	Duplicate rows removed
status	STRING	SUCCESS or FAILED for BATCH rows; PASS or FAIL for the GATE_1 row
error_message	STRING	Error text for FAILED rows; Gate 1 JSON for the GATE_1 row
started_at, finished_at	TIMESTAMP	Run start and end

5.3 ops.ops_watermark

Column	Type	Purpose
signal	STRING	One row per signal (gps, engine, fuel, temperature, device_status)
watermark_utc	STRING	Upper bound of the last successful run, ISO text with microseconds. 1900-01-01T00:00:00Z means nothing processed yet
updated_at	TIMESTAMP	When the watermark last changed

6. Data-quality reference

6.1 dq_flags values

Flag	Meaning	Set by
UNIT_CONVERTED	Temperature received in Fahrenheit and converted to Celsius	fleet_shared
SENSOR_NULL	The sensor reported no value	fleet_shared
SENSOR_FAULT	The sensor status reports a fault	fleet_shared
LOW_ACCURACY	GPS accuracy is poor	fleet_shared
SCHEMA_DRIFT	The message differs from the expected schema but is still usable	fleet_shared
SPIKE	Temperature jumped from the previous reading by more than the configured limit	notebook (section 3.9)
FLATLINE	Temperature sensor repeated the same value for the configured number of readings	notebook (section 3.9)
IMPOSSIBLE_JUMP	GPS position moved faster than the configured speed limit	notebook (section 3.9)

Several flags on one row are stored in one text value separated by commas. Values present in the Silver tables after the current run: gps: SCHEMA_DRIFT (13 rows); temperature: UNIT_CONVERTED (304), SENSOR_NULL,SENSOR_FAULT (54), SCHEMA_DRIFT (4); engine and fuel: no flags.

6.2 DLQ error codes

Error code	Meaning
PARSE_ERROR	The raw text is not valid JSON
MISSING_REQUIRED_FIELD	A required field of the signal is missing
TYPE_MISMATCH	A field has the wrong data type
OUT_OF_RANGE	A value is outside the allowed range (including fuel level above the tank capacity)
INVALID_GPS	Latitude or longitude is not a valid position
INVALID_TIMESTAMP	The event time cannot be parsed or is in the future compared with the simulated now
UNKNOWN_VEHICLE	The vehicle is not in the reference data, or is not active on the event date
UNKNOWN_TRIP	The trip is not in the reference data
UNKNOWN_SIGNAL_TYPE	The signal type is not supported
SCHEMA_DRIFT	The schema version is not supported

7. Results of the current run

Smoke run: 74,194 Bronze messages. Every signal satisfies bronze = silver + dlq + duplicates removed.

7.1 Counts per signal

Signal	Bronze	Silver	DLQ	Duplicates removed	Balanced	Invalid share
gps	30,895	30,472	275	148	True	0.890%
engine	30,926	30,512	271	143	True	0.876%
fuel	5,176	5,105	30	41	True	0.580%
temperature	7,197	7,105	51	41	True	0.709%
Total	**74,194**	**73,194**	**627**	**373**	True	**0.845%**

7.2 DLQ rows by error code

Error code	gps	engine	fuel	temperature	Total
PARSE_ERROR	73	51	14	20	158
MISSING_REQUIRED_FIELD	53	74	6	12	145
TYPE_MISMATCH	51	56	8	11	126
OUT_OF_RANGE	35	36	0	0	71
INVALID_GPS	23	0	0	0	23
INVALID_TIMESTAMP	21	27	1	1	50
UNKNOWN_VEHICLE	13	22	1	5	41
SCHEMA_DRIFT	6	5	0	2	13
Total	**275**	**271**	**30**	**51**	**627**

7.3 lateness_sec per Silver table (seconds)

Table	Rows	Non-NULL	Min	Median	p99	Max
silver_gps	30,472	30,472	1.138	1.552	1.783	2.562
silver_engine	30,512	30,512	1.138	1.552	1.784	2.047
silver_fuel	5,105	5,105	1.656	1.807	1.939	2.000
silver_temperature	7,105	7,105	1.150	1.761	1.801	1.869

All values are positive and within a few seconds, which is the expected size of the ingestion delay of a clean smoke run.

8. Re-running and rebuilding

8.1 Why a re-run is safe

- **Watermark:** a signal whose window is empty prints "nothing to do" and writes nothing. The watermark moves only after a successful run.
- **DLQ:** the DLQ rows of the same signal and window are deleted before new ones are written.
- **MERGE:** only event_ids that are not in Silver are inserted.
- **Failure:** an exception logs a FAILED row and leaves the watermark unchanged, so the same window is processed again.

8.2 Rebuilding Silver from Bronze

Silver is derived completely from Bronze, which is never modified. To rebuild it, empty the Silver tables, clear the batch log and DLQ rows and reset the watermarks, then run the function cells and `main()` again. These statements are run in a cell of their own:

```
for s in ["gps", "engine", "fuel", "temperature"]:
    spark.sql(f"DELETE FROM silver.silver_{s}")
spark.sql("DELETE FROM ops.dlq_events WHERE stage = 'BATCH'")
spark.sql("""DELETE FROM ops.ops_ingestion_log
             WHERE signal IN ('gps','engine','fuel','temperature') OR stage = 'GATE_1'""")
spark.sql("""UPDATE ops.ops_watermark SET watermark_utc = '1900-01-01T00:00:00Z',
             updated_at = current_timestamp()
             WHERE signal IN ('gps','engine','fuel','temperature')""")
```

Emptying Silver without resetting the watermarks leaves Silver empty, because the job then finds no new Bronze rows. Both actions are always done together.

9. Function reference

Function	Purpose
`load_manifest, check_manifest_against_bronze`	Read run_manifest.json; compare run_summary.json with Bronze counts
`get_window(signal)`	Return the (lower, upper) Bronze ingestion-time window
`load_lookups()`	Known vehicles, known trips and tank capacity per vehicle
`make_validate_udf(...)`	Spark UDF around fleet_shared.validate_message
`simulated_now(upper)`	Simulated time at the upper bound, for the future-timestamp check
`load_vehicle_intervals, split_by_vehicle_activity`	Vehicle-active-on-event-date check
`validate(signal, lower, upper, lookups)`	Read the window, validate, split into good and bad rows
`write_dlq(bad, signal, batch_id, lower, upper)`	Idempotent write to ops.dlq_events
`parse_and_enrich(good, signal)`	Typed columns, dq_flags text, trip enrichment, lateness_sec
`dedup_batch(enriched)`	One row per event_id; returns rows kept and duplicates removed
`add_sequence_flags(df, signal)`	SPIKE, FLATLINE and IMPOSSIBLE_JUMP
`merge_silver(deduped, signal)`	MERGE into silver.silver_<signal>
`write_log, set_watermark`	Ingestion log row; watermark update
`process_signal(signal)`	All steps for one signal
`run_gate1, write_gate_log`	Gate 1 status and its log row
`main()`	All signals, balance table and Gate 1